

Resultado de aprendizaje U2

```
export default function App() {  
  const [tasks, setTasks] = useState<Task[]>(initialTasks);  
  const [newTask, setNewTask] = useState<string>('');  
  const [editingTaskId, setEditingTaskId] = useState<string | null>(null);  
  const [taskToEdit, setTaskToEdit] = useState<string>('');  
  
  function handleAddTask() {  
    if (newTask.trim().length === 0) {  
      return;  
    }  
  
    const newTaskObj: Task = {  
      id: Math.random().toString(),  
      title: newTask,  
      done: false,  
      date: new Date(),  
    };  
  
    setTasks([...tasks, newTaskObj]);  
    setNewTask('');
```

Como primera función tenemos handleAddTask que esta función se encarga de agregar una nueva tarea a la lista de tareas. Primero, verifica si el campo de entrada no está vacío. Si hay texto en el campo de entrada, crea un nuevo objeto de tarea con un ID único, el título de la tarea, un estado de "no completada" y la fecha actual. Luego, actualiza el estado de la lista de tareas para incluir esta nueva tarea y limpia el campo de entrada. Y el evento aquí es cuando el usuario presiona el botón de agregar.

```
    }  
  
    ✓ function handleDeleteTask(id: string) {  
      const updatedTasks = tasks.filter(task => task.id !== id);  
      setTasks(updatedTasks);  
    }
```

En la segunda función tenemos a handleDeleteTask que esta función elimina una tarea específica de la lista de tareas. Filtra la lista de tareas, eliminando la tarea cuyo ID coincide con el ID pasado como argumento, y actualiza el estado de la lista de tareas

con las tareas restantes. El evento de esta función es cuando el usuario presiona el botón de eliminar de una tarea específica.

```
function handleToggleTask(id: string) {  
  const updatedTasks = tasks.map(task =>  
    task.id === id ? {...task, done: !task.done} : task,  
  );  
  setTasks(updatedTasks);  
}
```

En la tercera función tenemos al `handleToggleTask` que básicamente esta función cambia el estado de una tarea específica entre completada y no completada. Recorre la lista de tareas y, si encuentra la tarea cuyo ID coincide con el ID pasado como argumento, cambia su estado de "done" al valor opuesto. Luego, actualiza el estado de la lista de tareas con esta modificación. Donde el evento es cuando el usuario presiona el botón de hecho o deshacer de una tarea específica.

```
function handleEditTask(id: string, newTitle: string) {  
  const updatedTasks = tasks.map(task =>  
    task.id === id ? {...task, title: newTitle} : task,  
  );  
  setTasks(updatedTasks);  
  setEditingTaskId(null);  
  setTaskToEdit('');  
}
```

Como cuarta función tenemos a `handleEditTask`. Esta función guarda los cambios realizados en el título de una tarea específica. Recorre la lista de tareas y, si encuentra la tarea cuyo ID coincide con el ID pasado como argumento, actualiza su título con el nuevo título proporcionado. Luego, actualiza el estado de la lista de tareas y restablece los estados relacionados con la edición. Su evento es cuando el usuario presiona el botón de guardar después de editar una tarea.

```
function renderItem({item}: {item: Task}) {  
  return (  
    <View style={[styles.listItem, item.done && styles.completedTask]}>  
      {editingTaskId === item.id ? (  
        <>  
          <TextInput  
            style={styles.textInput}  
            value={taskToEdit}  
            onChangeText={setTaskToEdit}  
          />  
          <TouchableOpacity  
            style={styles.saveButton}  
            onPress={() => handleEditTask(item.id, taskToEdit)}>  
            <Text style={styles.saveText}>Guardar</Text>  
          </TouchableOpacity>  
        </>  
      ) : (  
        <>
```

Como última función tenemos a `renderItem({item}: {item: Task})`. Esta función nos permite renderizar cada tarea en la lista de tareas y verifica si la actual está en modo de edición. Si es así, muestra un campo de entrada y un botón para guardar los cambios. Si no está en modo de edición, muestra el título de la tarea y los botones para marcarla como completada/no completada, editarla y eliminarla. En el cual su evento se utiliza internamente por el componente `FlatList` para renderizar cada elemento de la lista de tareas.

En conclusión, Estos eventos y funciones en conjunto permiten que los usuarios gestionen una lista de tareas en la aplicación móvil, añadiendo, eliminando, completando y editando tareas según sea necesario.